



Semaine 5 : Les tableaux

Objectif : Stocker plusieurs valeurs dans une seule variable **Durée** : 2 heures **Lien Scratch** :
Les listes 



Ce que tu vas apprendre

1. Créer un tableau
2. Accéder aux éléments par leur position (index)
3. Ajouter / retirer des éléments
4. Parcourir un tableau avec `for` et `forEach`
5. Chercher dans un tableau

1 Pourquoi un tableau ?

Imagine que tu veux stocker l'inventaire d'un joueur Minecraft. Sans tableau, il faudrait UNE variable par bloc :

```
let bloc1 = "terre";  
let bloc2 = "pierre";  
let bloc3 = "diamant";  
// ... beaucoup trop long !
```

Avec un tableau, **une seule variable** suffit :

```
let inventaire = ["terre", "pierre", "diamant"];
```

Scratch vs JavaScript

```
■ Liste : inventaire  
1. terre  
2. pierre  
3. diamant
```

```
let inventaire = [  
  "terre",  
  "pierre",  
  "diamant"  
];
```

2 Créer un tableau

```
// Tableau de strings  
let pseudos = ["Steve", "Alex", "Herobrine"];  
  
// Tableau de nombres  
let scores = [10, 25, 7, 100];  
  
// Tableau vide (pour le remplir plus tard)  
let inventaire = [];
```

 Exercice : exercice_1_creer_tableau.js

3 Accéder à un élément

Chaque case a un **numéro** appelé **index**. ⚠ **Attention** : ça commence à **0**, pas à 1 !

```
let pseudos = ["Steve", "Alex", "Herobrine"];  
//           ↑0       ↑1       ↑2  
  
console.log(pseudos[0]); // "Steve"  
console.log(pseudos[2]); // "Herobrine"
```

Combien d'éléments dans un tableau ?

La propriété `.length` te donne la taille.

```
let pseudos = ["Steve", "Alex", "Herobrine"];
console.log(pseudos.length); // 3
```

Le **dernier** élément a donc l'index `length - 1`.

Modifier un élément

```
let pseudos = ["Steve", "Alex", "Herobrine"];
pseudos[1] = "Notch";
console.log(pseudos); // ["Steve", "Notch", "Herobrine"]
```



Exercice : `exercice_2_acceder_modifier.js`

4 Ajouter et retirer

Méthode	Effet
<code>tab.push(x)</code>	Ajoute <code>x</code> à la fin
<code>tab.pop()</code>	Retire le dernier élément
<code>tab.unshift(x)</code>	Ajoute <code>x</code> au début
<code>tab.shift()</code>	Retire le premier élément

```
let inventaire = ["terre"];

inventaire.push("pierre");      // ["terre", "pierre"]
inventaire.push("diamant");    // ["terre", "pierre", "diamant"]
inventaire.pop();              // ["terre", "pierre"]
```

 **Exercice :** `exercice_3_push_pop.js`

5 Parcourir un tableau avec `for`

Pour faire quelque chose avec **chaque** élément, on boucle.

```
let inventaire = ["terre", "pierre", "diamant"];

for (let i = 0; i < inventaire.length; i++) {
  console.log(`Case ${i} : ${inventaire[i]}`);
}
```

 **Exercice :** `exercice_4_parcourir_for.js`

6 Parcourir avec `forEach`

Plus court quand on n'a pas besoin de l'index :

```
let inventaire = ["terre", "pierre", "diamant"];

inventaire.forEach(function(bloc) {
  console.log(bloc);
});
```

`bloc` prend tour à tour la valeur de chaque case.

 Exercice : `exercice_5_foreach.js`

7 Chercher dans un tableau

```
let inventaire = ["terre", "pierre", "diamant"];

inventaire.includes("diamant"); // true
inventaire.includes("or");      // false

inventaire.indexOf("pierre");   // 1
inventaire.indexOf("or");       // -1 (pas trouvé)
```

 Exercice : `exercice_6_recherche.js`

8 Tout combiner

 Exercice : `exercice_7_inventaire_minecraft.js`

Défi de la semaine

 Défi : `defi_liste_courses.js`

Crée une liste de courses interactive : ajouter, retirer, afficher.

À retenir

- Index commence à `0`
- `.length` donne le nombre d'éléments

- `push` / `pop` à la fin, `unshift` / `shift` au début
- `for` classique ou `forEach` pour parcourir
- `includes` pour tester la présence